

第4章 存储器

4.1 概述

4.2 主存储器

4.3 高速缓冲存储器



七、提高访存速度的措施

4.2

- 提高访存速度的必要性
 - 随着计算机应用领域的不断扩大，处理的信息量越来越多，对存储器的工作速度和容量要求越来越高
 - 因为CPU的功能不断增强，I/O设备的数量不断增多，致使主存的存取速度已成为计算机系统的瓶颈，提高访存速度成为迫不及待的任务

提高访存速度的措施

- 采用高速元器件 TTL存储器速度大于CMOS存储器速度
- 采用层次结构 用 Cache-主存 层次应对速度问题
- 调整主存结构

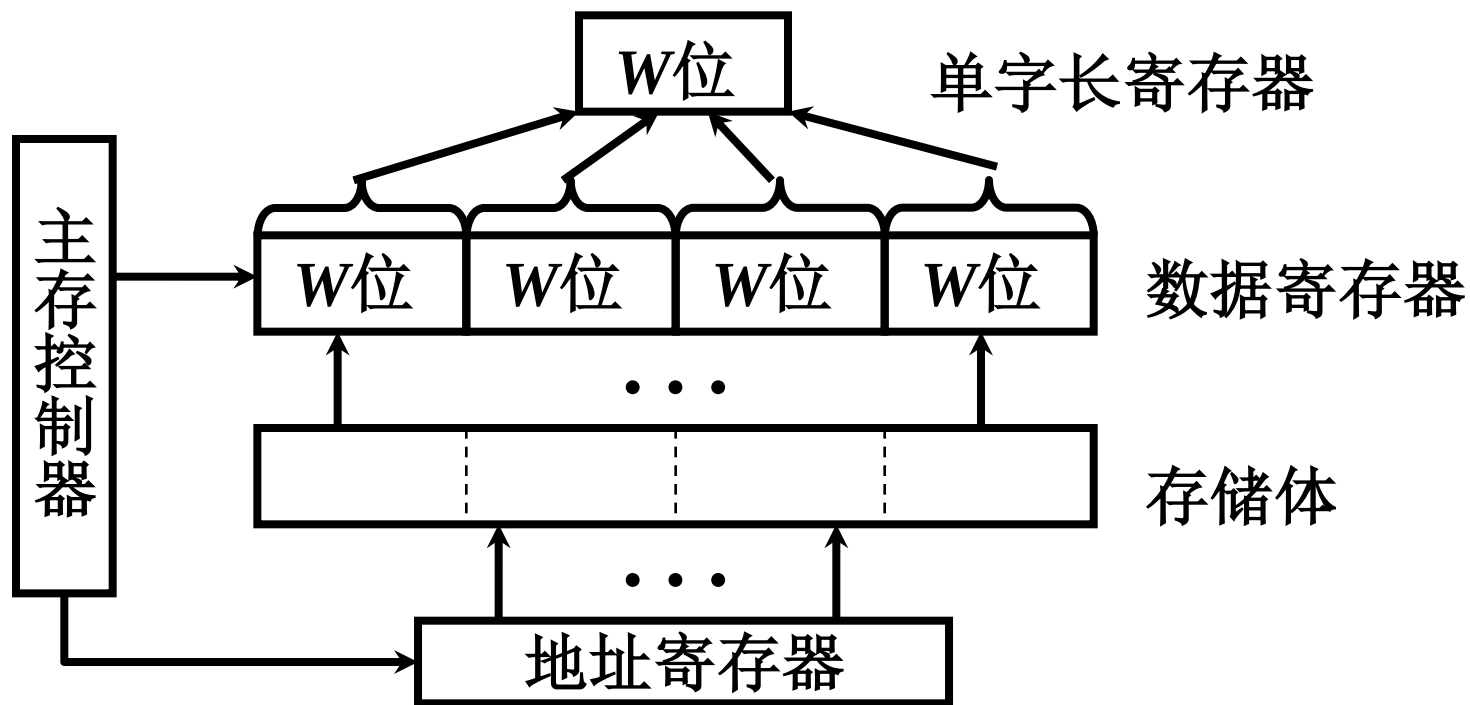
1. 单体多字系统

只有一个存储体，在一个存取周期内，从同一个地址，取出地址连续的多个字的指令或数据（暂时存放在内部寄存器中），然后逐条传送给CPU，相当于在一个存取周期内，主存向CPU传送多条指令，从而增加存储器带宽，提高了单体存储器的工作速度。



单体多字系统

单体四字结构存储器，在一个存取周期内，从同一地址取出地址连续的 $4 \times W$ 位的指令或数据



主存带宽提高多倍
指令和数据必须连续存放



单体多字系统要求一个存取周期内、一个存储体输出多个字，这实际上要求存取周期变短；多体并行系统是多个存储体在一个存取周期内并行工作，每个存储体的存取周期没有改变，这在存储器技术上难度更低

2. 多体并行系统

- ✓ 采用多体模块组成存储器，每个模块有相同的容量和存取速度
- ✓ 各模块（存储体）都有独立的地址寄存器、数据寄存器、地址译码驱动电路和读写电路，各模块能并行或者交叉工作



(1) **并行工作**，即同时访问多个存储器模块，同时启动，同时读出，多个存储体并行工作，同时读出的多个字在总线上分时传送

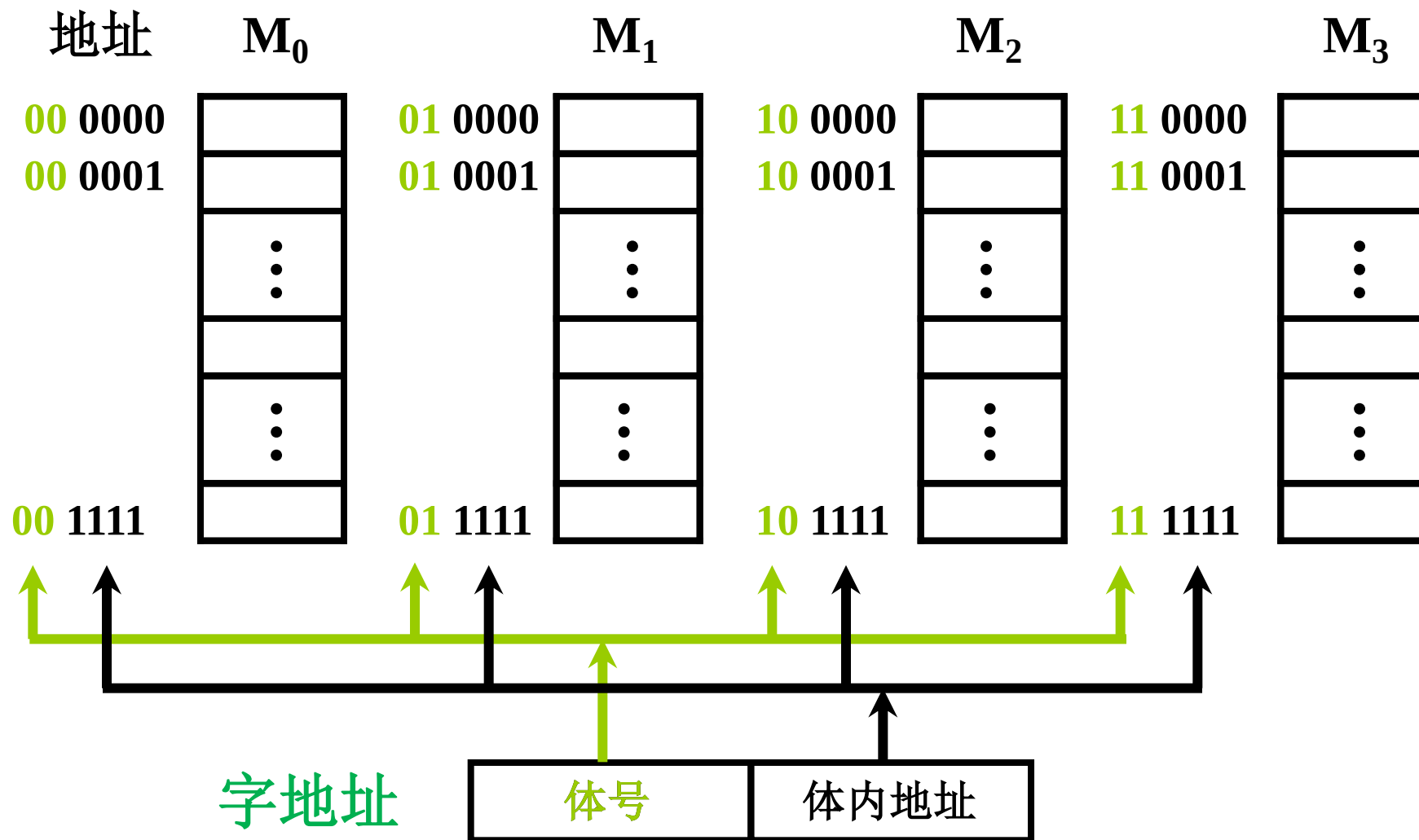
✓这种情况下，程序**按体内地址**顺序存放，存满一个体，再存于下一个体，也称**顺序存储**

✓此时，主存字地址属于**高位交叉编址**，即高位地址代表存储体号

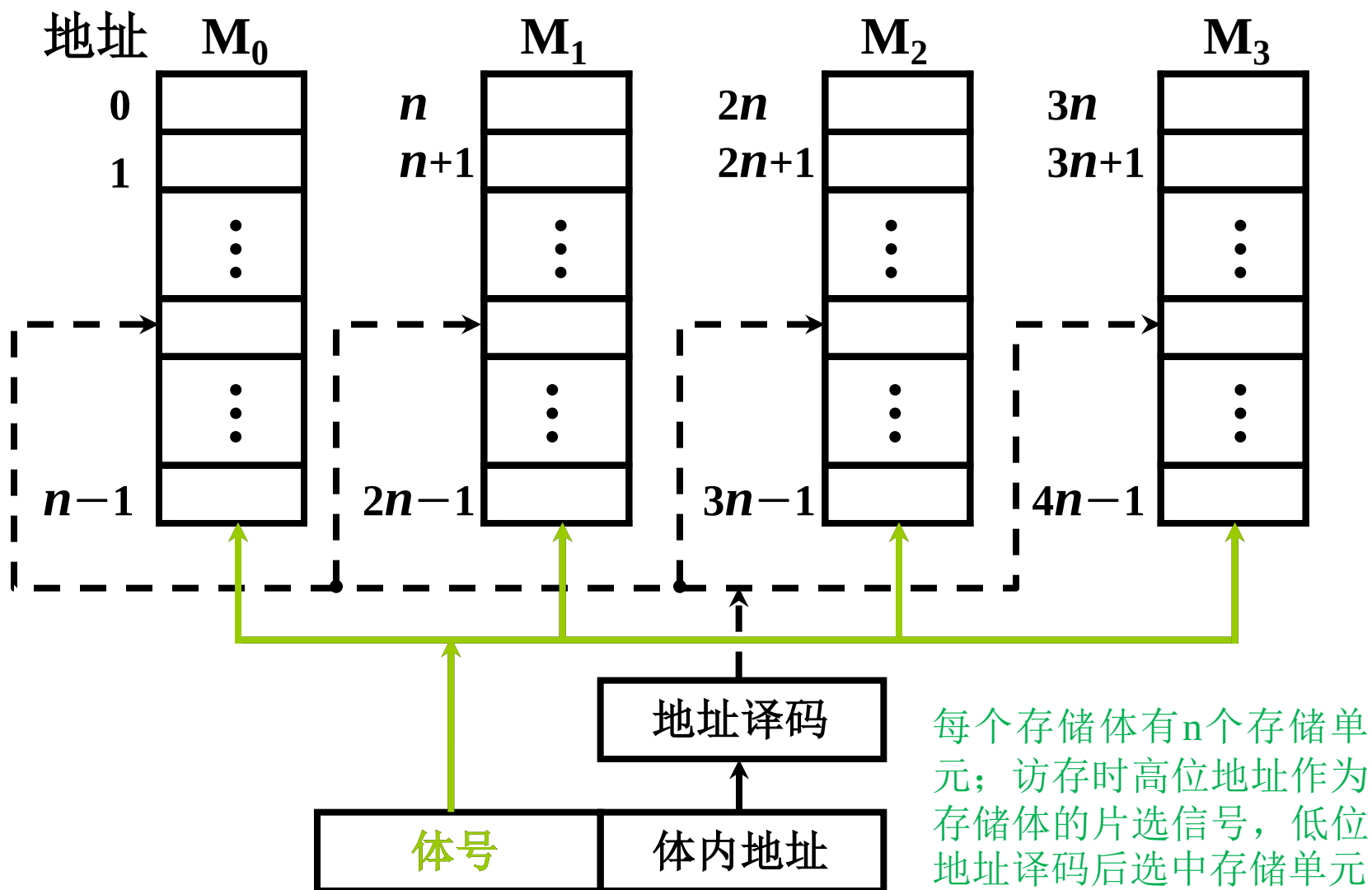
(2) **交叉工作**，程序连续存放在相邻的存储体中，从而使得字地址的低位表示存储体号，称为**低位交叉编址**



(1) 高位交叉编址 也称顺序存储，按地址顺序存满一个体，再存于下一个体



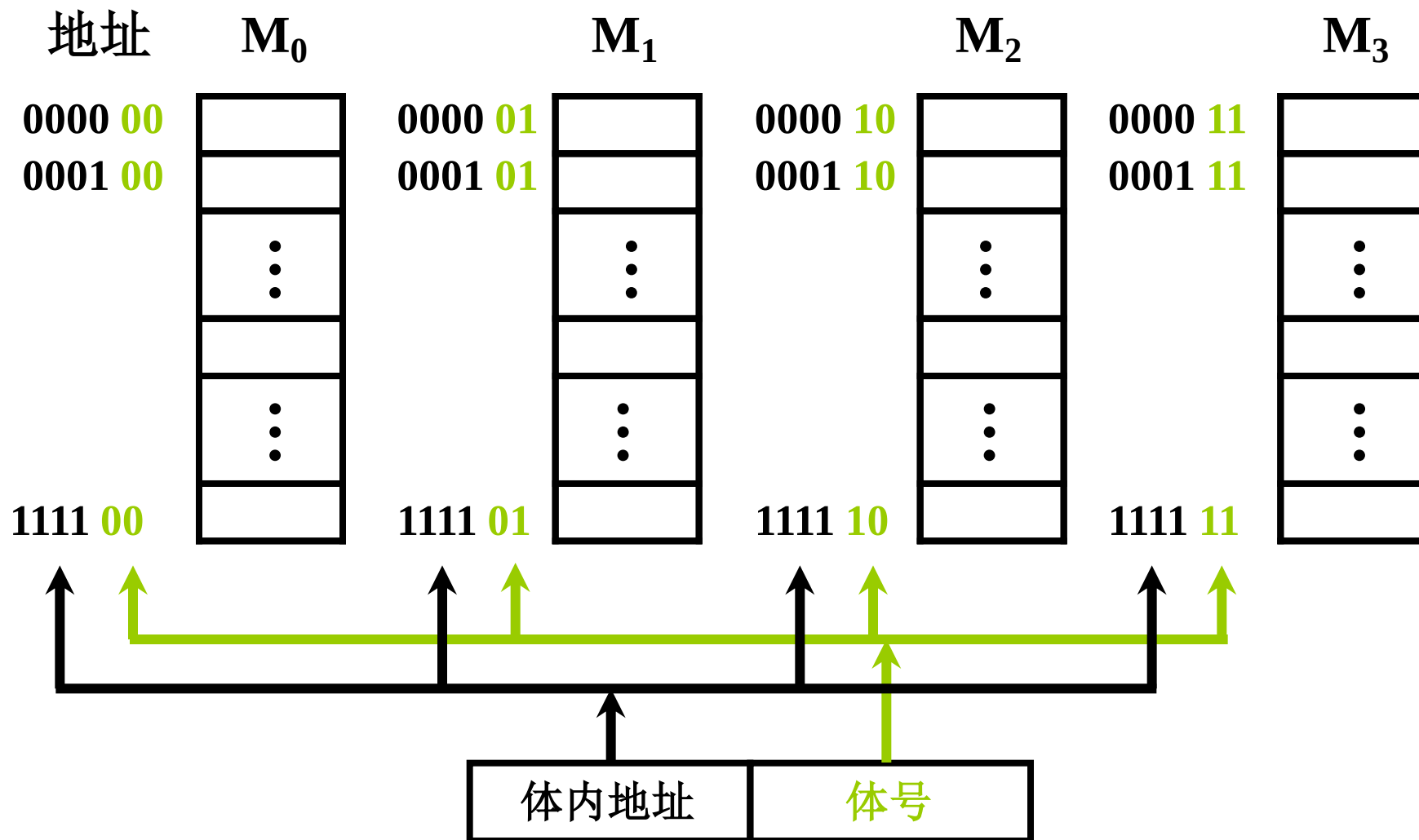
高位交叉编址 各个体并行工作



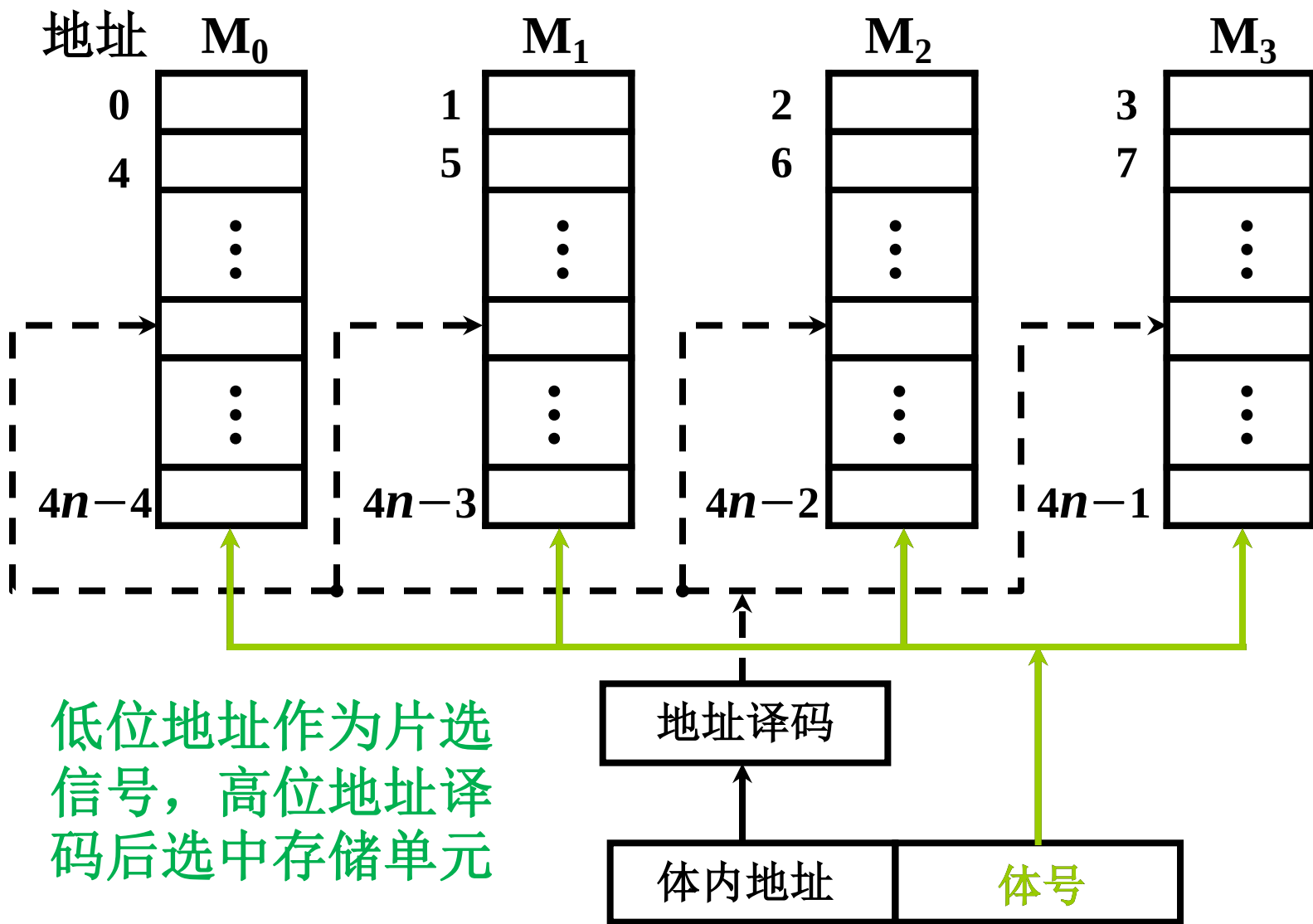
- ✓编址特点：高位地址表示存储体的体号，低位地址表示体内的存储单元地址
- ✓通过合理调度，能够使不同的访存请求源（CPU、外设等）访问不同的存储体，便可实现并行
- ✓由于体内地址是连续的，便于存储器的扩充

(2) 低位交叉编址

程序连续存放在相邻体中，
又称交叉存储

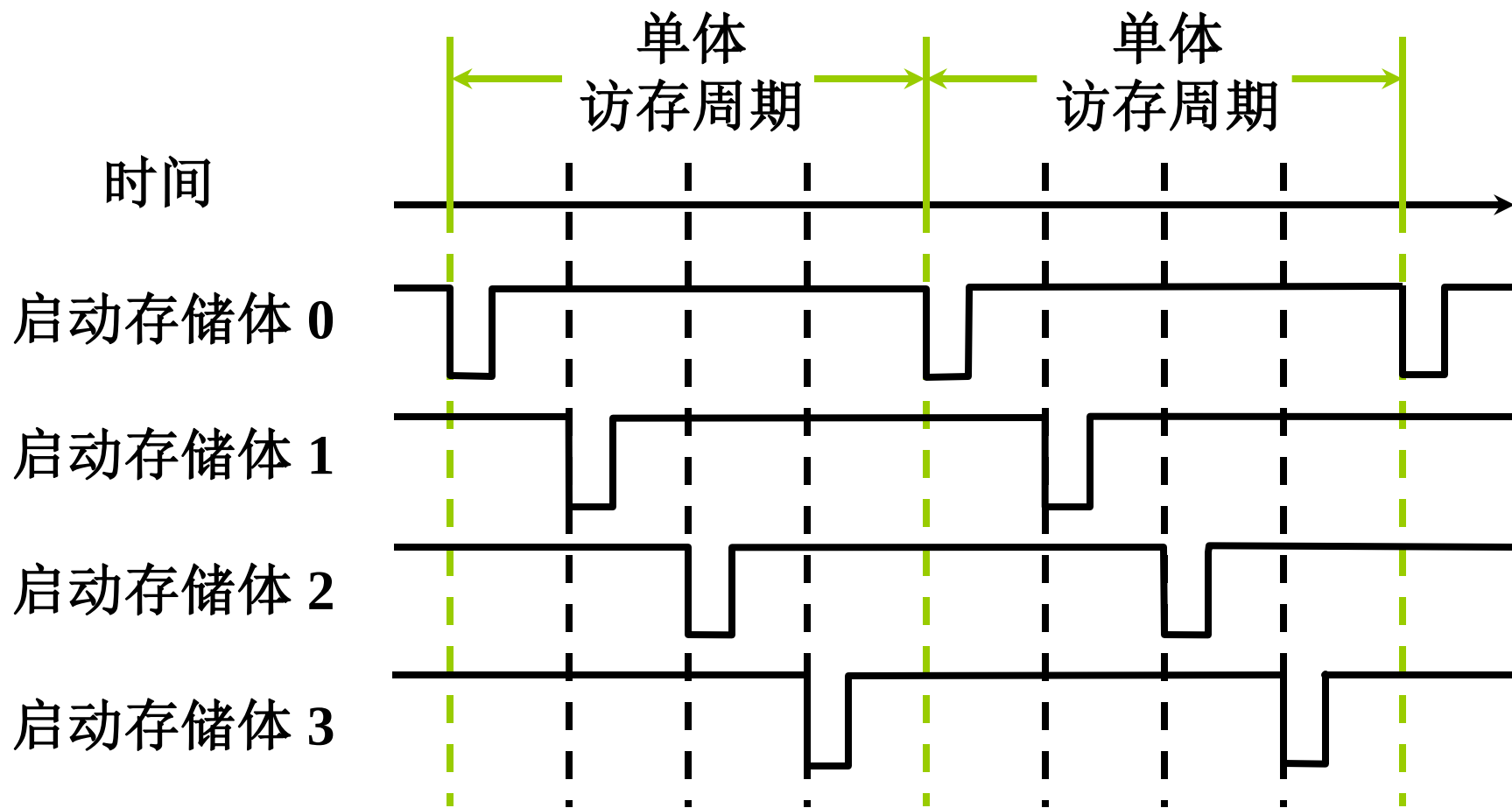


低位交叉编址 存储体个数一般为 2^n

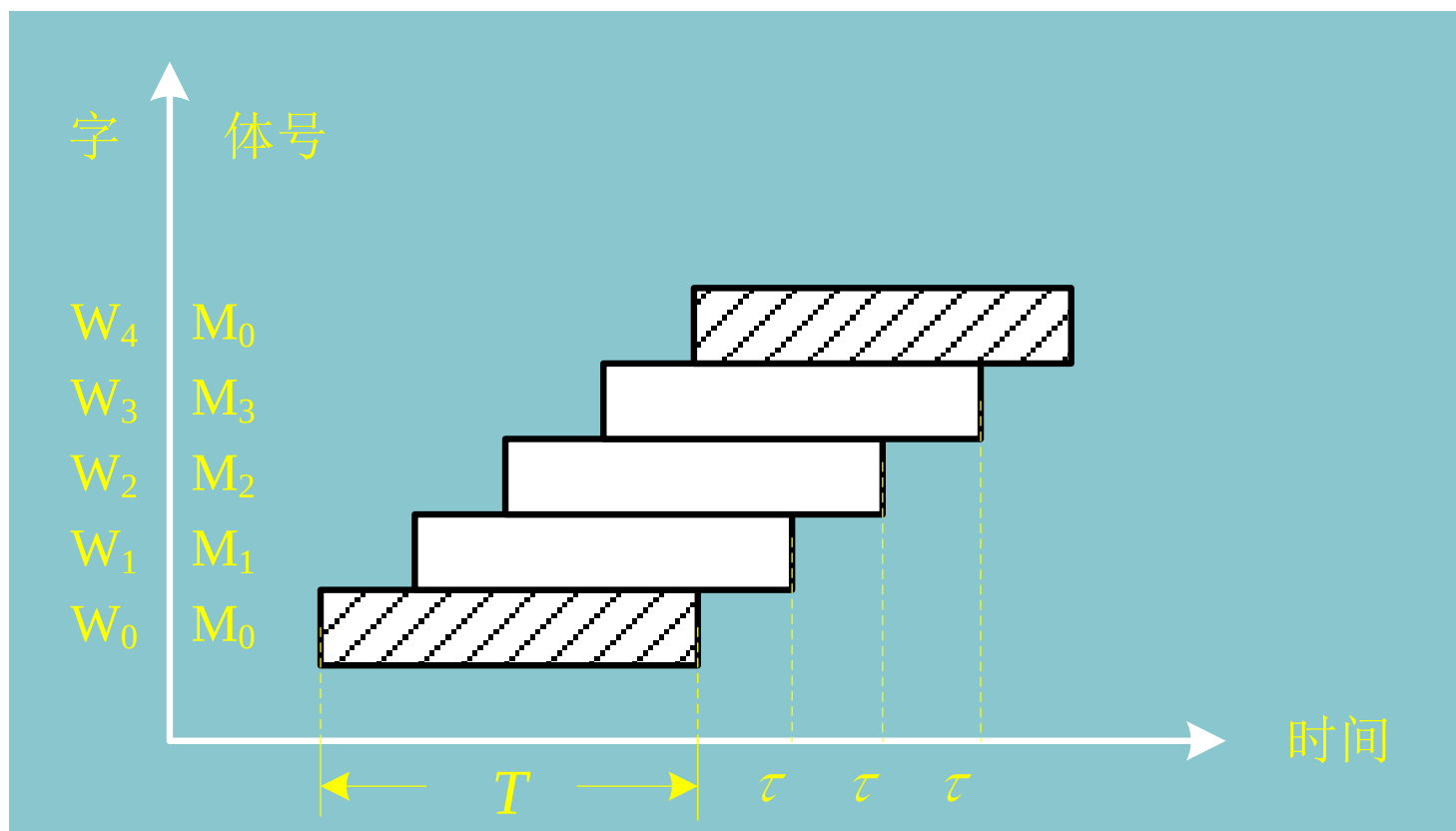


低位交叉的特点

CPU（取指令时）交叉访问各存储体，使各存储体的读写过程重叠进行，在一个存取周期时间内，低位交叉多体存储器能够向CPU提供多个存储字，实现在不改变每个体存取周期的前提下，增加存储器的带宽



设四体低位交叉存储器，存取周期为 T ，总线传输周期为 τ ，为实现流水线方式存取，应满足 $T = 4\tau$
注：总线速度需要快于各存储体速度



连续读取 4 个字所需的时间为 $T + (4 - 1)\tau$

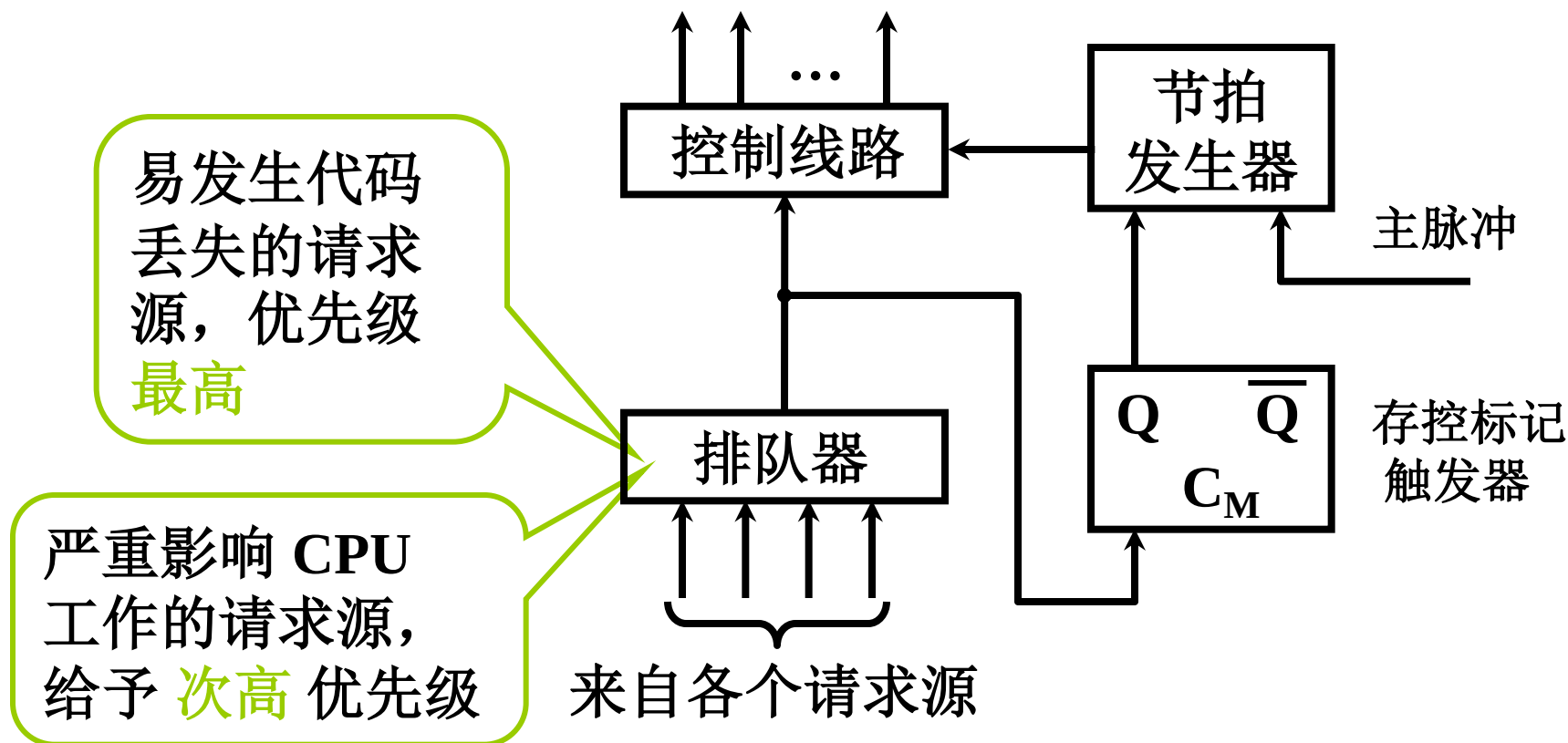


(3) 存储器控制部件（简称存控）

- 由于多体模块存储器要与CPU、辅存、外部设备等多部件交换信息，因此需要有一个存储器控制部件，决定在某一时刻主存与哪个部件交换信息，存储器控制器具有合理安排各部件请求访存的顺序以及控制主存读写操作的功能
- 存储器控制器由排队器、标记触发器、节拍发生器、控制线路等组成

- 排队器确定请求源的优先级，优先级从高到低：
外设、写数、读数、读指令
- 存控标记触发器：一旦响应某请求源，触发器置1，
启动节拍发生器
- 节拍发生器：产生固定节拍，与机器主脉冲同步，
使控制线路按照一定时序发出信号
- 控制线路：基于排队器信号和节拍发生器的信号，
向存储器各部件发出各种控制信号，实现对总线
控制以及完成存储器读写操作，并向访存请求源
发出回答信号，表示存储器已经响应请求

存储器控制部件结构图



3.高性能存储芯片

(1) SDRAM (同步 DRAM)

采用高性能存储芯片，也属于使用调整主存结构提高访存速度的措施，这通过设计对基本RAM结构的增强功能，提高DRAM的性能

- SDRAM与常用的异步RAM相比，不影响CPU的执行速度
- SDRAM与处理器的数据交换同步于系统的时钟信号，在系统时钟的控制下进行读出和写入
- CPU给出的地址和控制信号会被SDRAM锁存，直到指定的时钟周期后再响应，此间CPU可执行其他任务，无须等待



(2) RDRAM

由 **Rambus** 公司开发，采用专门的**DRAM**和高性能芯片接口取代现有存储器接口，**利用高速总线连接多个RDRAM芯片**，提升存储器带宽

(3) 带 Cache 的 DRAM

- 在**DRAM** 的芯片内**集成**一个由 **SRAM** 组成的**缓存**，
可用来保存**DRAM**一行的内容，可快速取出该行中任一“列组合”数据
- 连续改变列地址，可以从**SRAM**保存的一行中，连续读取各“列组合”数据，就是**猝发式读取**



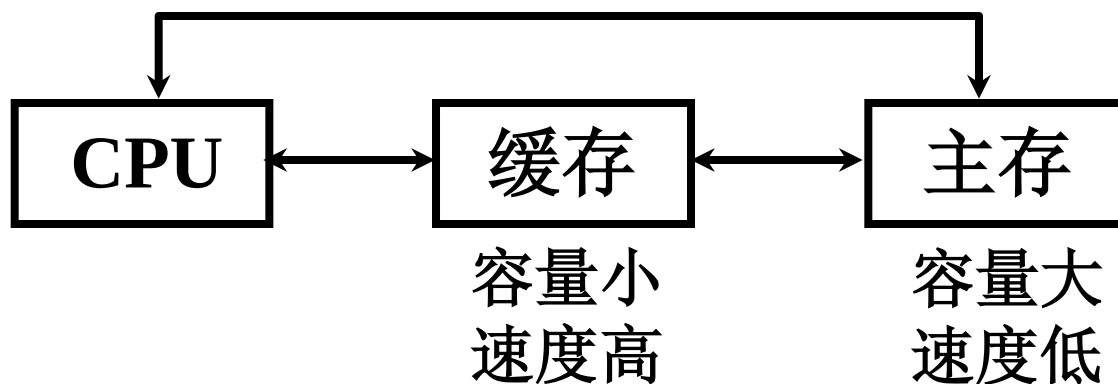
4.3 高速缓冲存储器

一、概述

1. 为什么需要缓存

主存把CPU要访问的信息提前送至缓存，CPU只需与高速缓存交换信息

- 避免 CPU “空等(I/O设备与CPU争抢访存)” 现象
- CPU 和主存（DRAM）的速度差越来越大



主存--Cache结构的合理性依据：程序访问的局部性原理



使用Cache的依据：程序访问的局部性原理

- “CPU从主存取指令或者数据，在一定时间内，只对主存局部区域进行访问”
- 原因：指令和数据在主存内是连续存放的，并且有些指令和数据会被多次调用
- 同一程序的指令、数据在主存的地址分布不是随机的，而是相对簇聚，使得CPU执行程序时，访存具有相对的局部性
- 只要将CPU“近期要用”的程序和数据提前从主存送到Cache，就可以让CPU在“一段时间内”只访问Cache，解决CPU和主存速度不匹配问题

2. Cache 的工作原理

- 假定主存包含 2^n 个可编址的字，每个字具有惟一的 n 位地址
- 主存和缓存都分成许多大小相同的块，每个块包含 $2^b = B$ 个字， B 为块长， b 为块内字地址位数
- 主存中字的地址分为两段：高 m 位块地址 + 低 b 位块内地址， $2^m = M$ 为主存内包含的字块数 存储容量/
块大小
- Cache 中字的地址也分为两段：高 c 位块地址 + 低 b 位块内地址， $2^c = C$ 为缓存内包含的字块数
- 由于 $C \ll M$ ，任何时候都只有一部分主存块处于缓存中，CPU 要从主存中读取一个字时，有两种可能：

1) 如果所需的字已经在Cache中，直接从Cache取走，这称为CPU访问Cache“命中”，CPU和Cache通常每次传送一个字

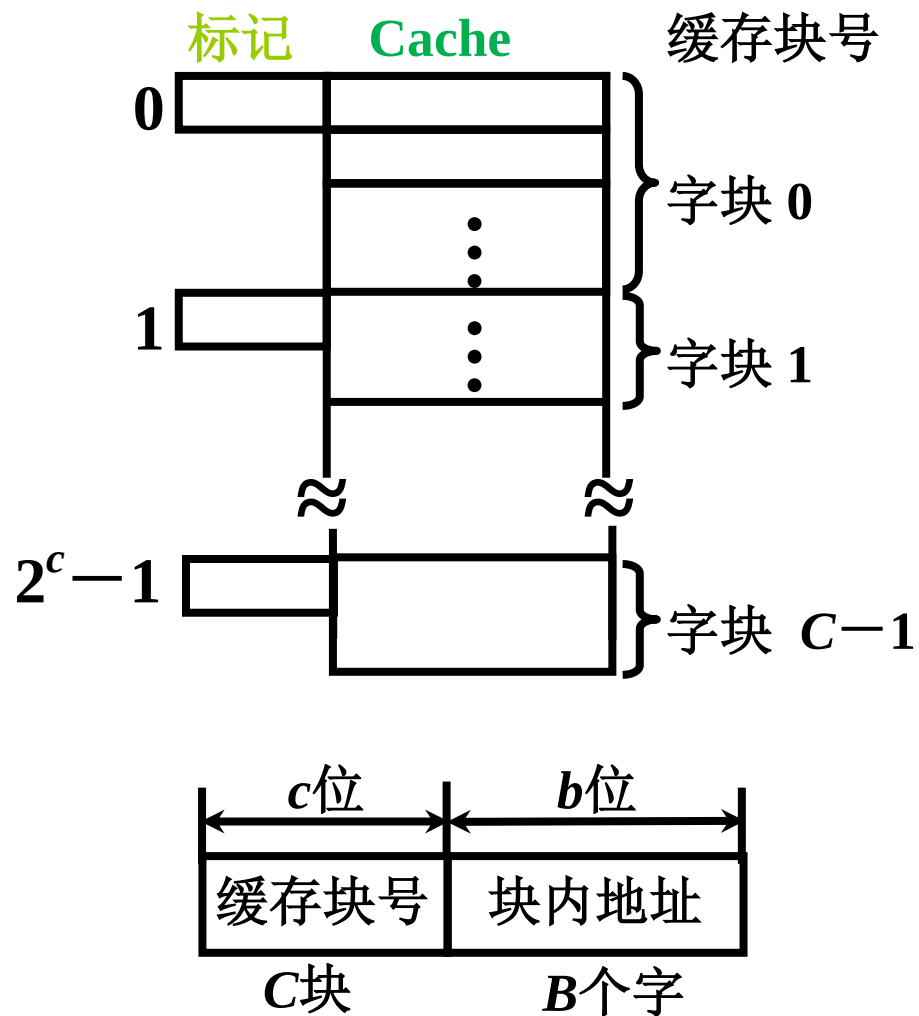
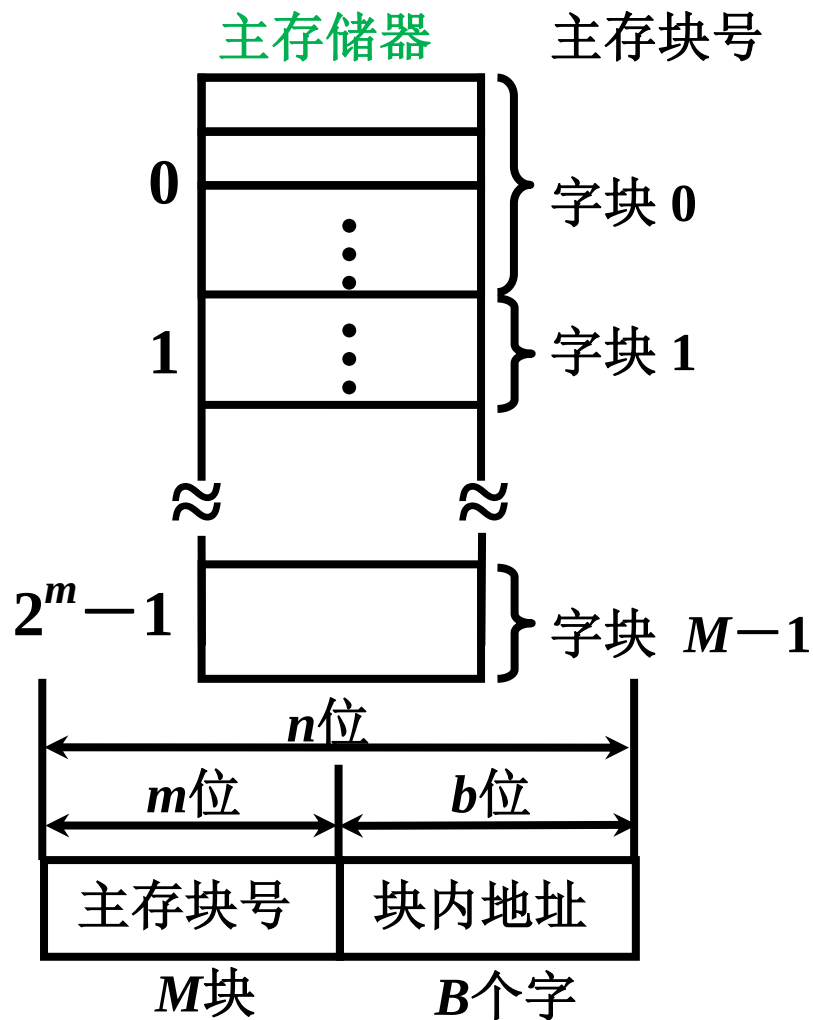
2) 如果所需的字不在Cache中，这称为CPU访问Cache“不命中”，这种情况从主存取字，并将该字所在的主存块调入缓存中

- 如果某主存块的内容已经调入相应的缓存块，则称该主存块与缓存块建立了对应关系
- 主存和Cache之间是字块传送，主存字块调入Cache时，不是在任意字块位置都可以存放

- 一个缓存块不能永久、唯一地对应一个主存块，因此需在每个缓存块设立一个标记，表示当前存放的是哪一个主存块，标记内容相当于主存块编号（高 m 位块地址或 m 位块地址的一部分）；
- 一个主存块可以对应特定一个Cache块、或全部Cache块中任一个字块、或某Cache分组中任一个块，这叫做主存—缓存地址映射；
- CPU访存时，给出的是存储字的主存地址（含主存块地址与块内地址），基于此，地址映射变换机构找到存储字所在主存块对应的Cache块，把Cache块中的主存字块标记和主存块编号进行比较，判断所需信息是否已经在缓存

2. Cache 的工作原理

(1) 主存和缓存的编址



主存和缓存按块组织

块的大小相同

B 为块长



(2) 命中与未命中

缓存共有 C 块

主存共有 M 块 $M \gg C$ ，访存有两种可能：

命中 所需字已在缓存中，所在主存块已调入缓存

主存块与缓存块 建立 了对应关系

某缓存块用标记保存与建立了对应关系的主存块号

未命中 所需字不在Cache内，该字所在主存块
未调入 缓存

主存块与缓存块 未建立 对应关系



(3) Cache 的命中率

CPU 访问的信息在 Cache 中的 比率

$$h = \frac{N_c}{N_c + N_m}$$

N_c 为访问 Cache 总命中次数, N_m 为访问主存总次数

命中率 与 Cache 的 容量 与 块长 有关

一般每块可取 4 ~ 8 个可编址字/字节, 或者:

块长取一个存取周期内从主存调出的信息长度

CRAY_1	16体交叉	块长取 16 个存储字
IBM 370/168	4体交叉	块长取 4 个存储字



(4) Cache –主存系统的效率

效率 e 与 命中率 有关

$$e = \frac{\text{访问 Cache 的时间}}{\text{平均访问时间}} \times 100\%$$

设 Cache 命中率为 h ，访问 Cache 的时间为 t_c ，
访问 主存 的时间为 t_m

$$\text{则 } e = \frac{t_c}{h \times t_c + (1-h) \times t_m} \times 100\%$$



3. Cache 的基本结构

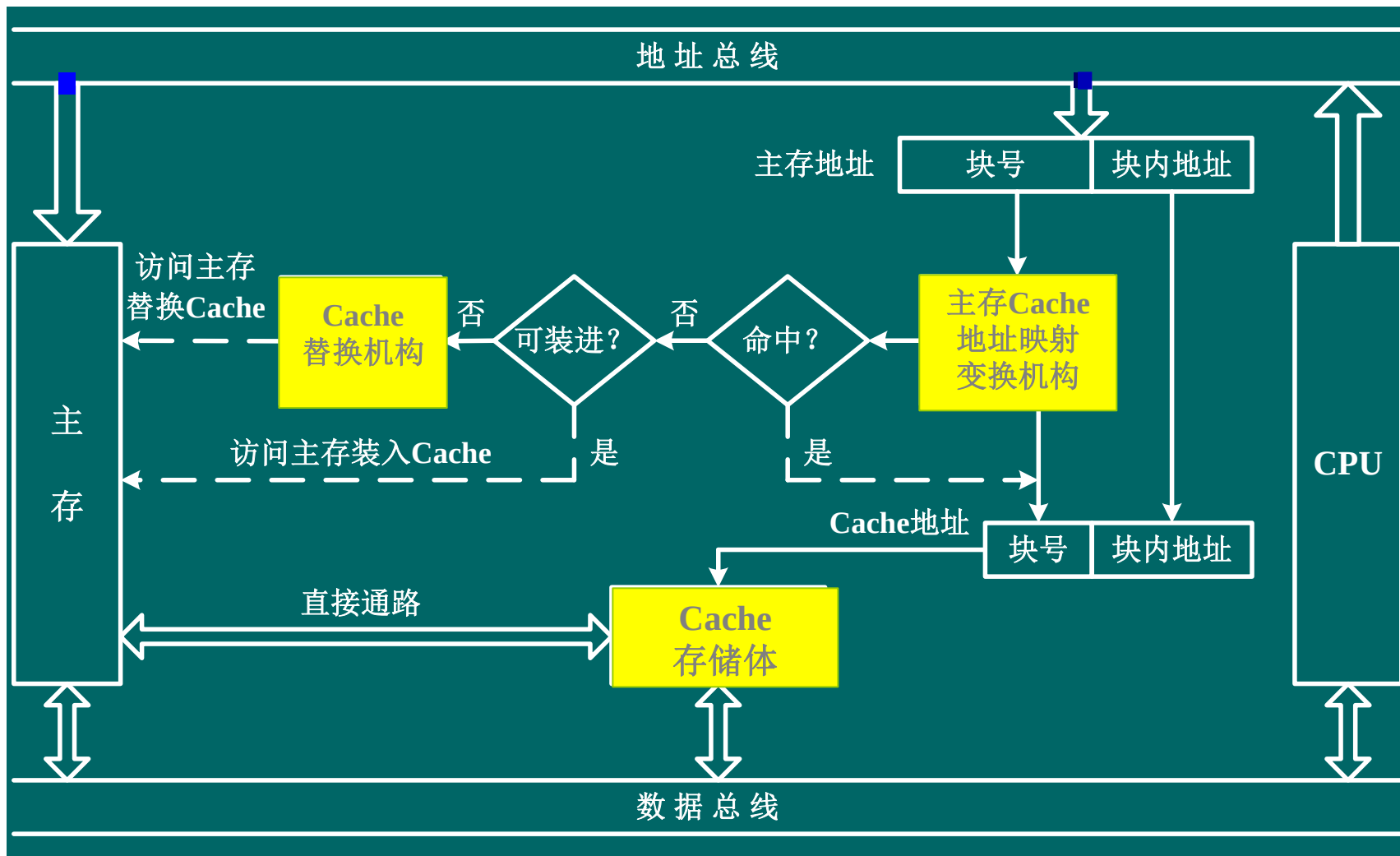
Cache主要由**存储体**、**地址映射变换部件**、**Cache替换机构**几大模块组成：

- **存储体**以块为单位和主存交换信息，以字为单位和CPU交换信息，Cache 访存优先级最高；
- **地址映射变换机构**把CPU送来的访问字的主存地址映射为其对应的Cache地址，由于存储字在两类块内的地址（低位地址）相同，所以主要是主存块号和Cache块号之间的转换，若转换后的Cache块和CPU欲访问的主存块建立了对应关系（查询该Cache块里的主存字块标记可知），则CPU直接访问Cache取走存储字；



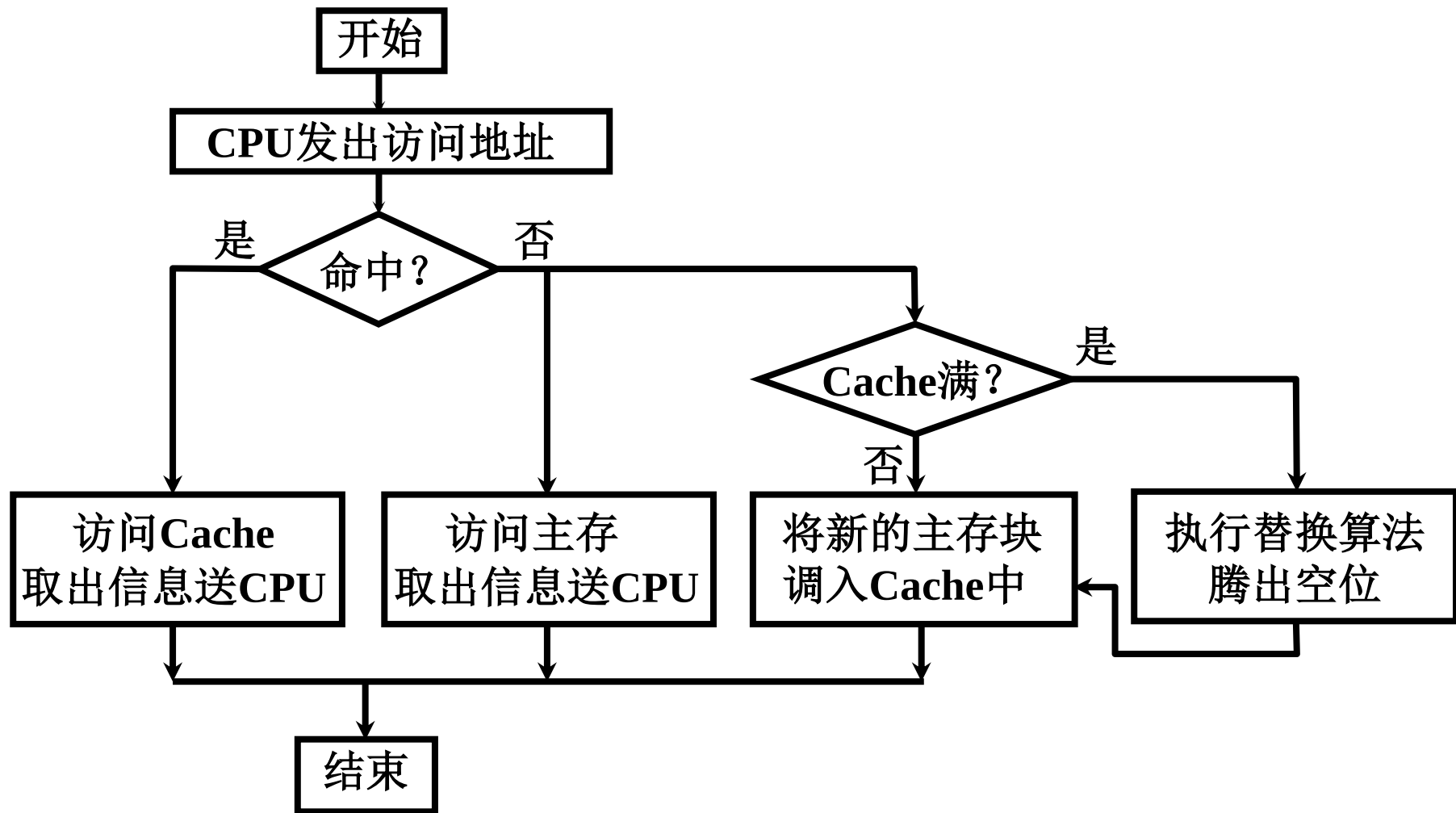
- 如地址映射后发现主存块和Cache块未建立对应关系，则访问主存取字，并将所在字块一并调入Cache中，若此时Cache已满，则进行Cache 替换；
- 替换机构根据替换算法确定移出哪个Cache字块返回主存，以装入即将调入的主存块。替换操作由硬件完成，对用户透明，用户不知道Cache中有哪些字块，用户编程使用主存字地址。

Cache 结构图



4. Cache 的 读写 操作

读



4. Cache 的 读写 操作

写 写操作复杂，要保持Cache 和主存内容一致

- **写直达法 (Write – through)**

写操作时数据既写入Cache又写入主存，始终一致

写操作时间就是访问主存的时间；读操作未命中时，被替换的块不必写回主存，替换策略比较容易实现

- **写回法 (Write – back)**

写操作时只把数据写入 Cache 而不写入主存

当 Cache 数据被替换出去时才写回主存

写操作时间就是访问 Cache 的时间

读操作的情况，若 Cache 失效发生数据替换时，被替换的块需写回主存，增加了 Cache 的复杂性



5. Cache 的改进 从早期的一个缓存发展到多个

(1) 增加 Cache 的级数

片载（片内）Cache /一级缓存

片外 Cache /二级缓存

(2) 统一缓存和分立缓存

指令 Cache 数据 Cache

与主存结构有关 指令、数据是否分开存储

与指令执行的控制方式有关 是否流水线控制

Pentium	8K 指令 Cache	8K 数据 Cache
---------	-------------	-------------

PowerPC620	32K 指令 Cache	32K 数据 Cache
------------	--------------	--------------



二、Cache – 主存的地址映射

- 由主存的字地址映射到Cache的字地址称为地址映射，有直接映射、全相联映射、组相联映射等映射方式

与一个缓存块对应的所有主存块可以用m-c位二进制代码索引，称为主存字块标记

1. 直接映射：

每个主存块只能与固定的一个缓存块对应；每个缓存块可以和多个主存块对应（也就是主存块数对缓存块数的倍数 $2^m / 2^c = 2^{m-c}$ ）

映射关系： $i = j \bmod 2^c$ i, j 是cache、主存块号

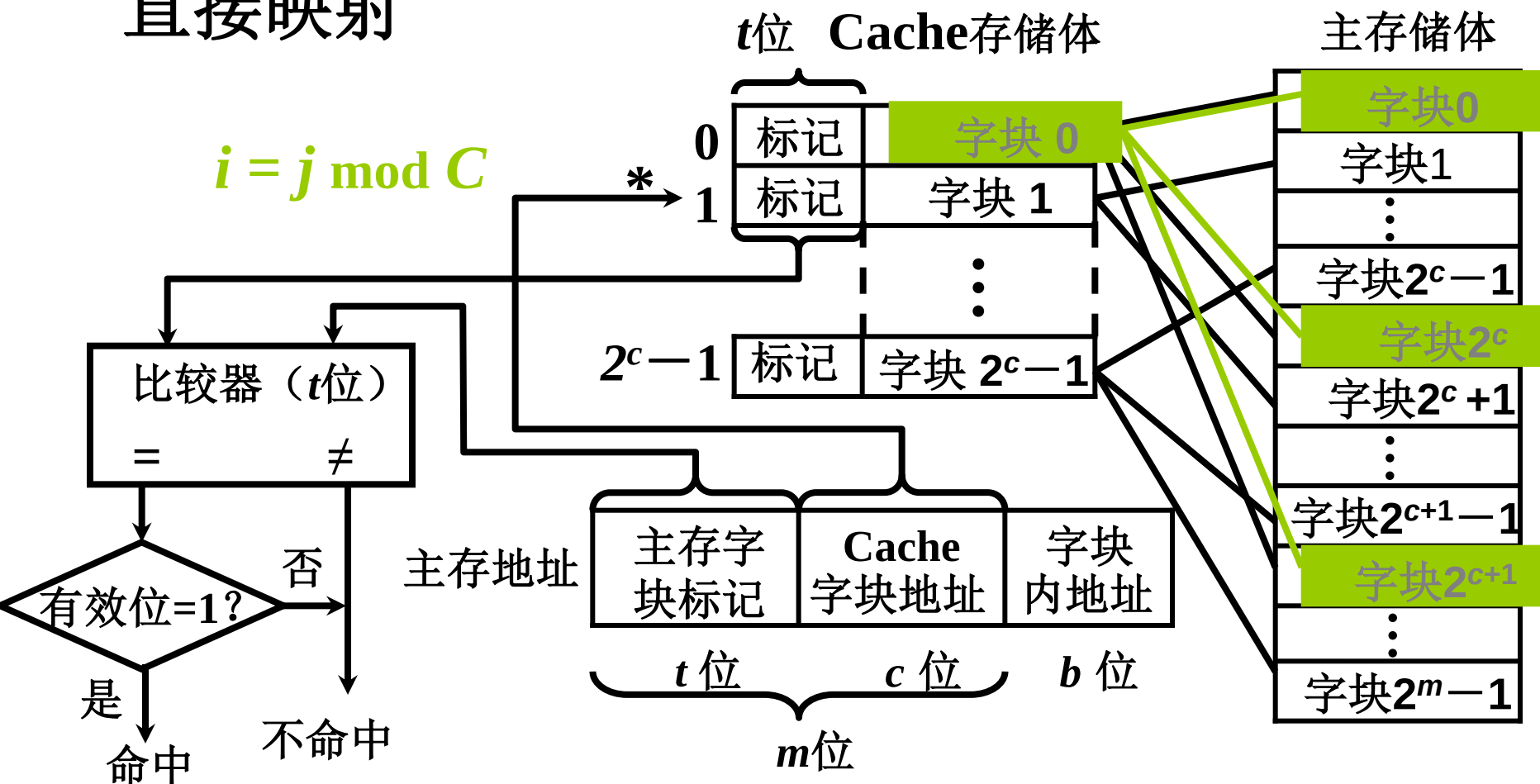
若主存块数为64，缓存块数是8，则第1号缓存块对应1, 9, 17, 25, 33, 41, 49, 57号主存块： $1+8*n$

缓存块地址由前面的映射关系得到。如前所述：每个 Cache 块对应 $64/8=8$ 个主存块； $m-c=3$ ，字块标记是3位二进制，正好可以用3位二进制来索引

- 主存字地址的构成：
 - 主存块地址(m 位)+块内地址 (b 位)
- 主存 m 位块地址又被分成两部分：
 - 主存字块标记 ($m-c$ 位) +Cache块地址 (c 位)
- CPU访存过程：Cache接到CPU送来的主存字地址，根据中间的 c 位找到对应的Cache块，再检查该Cache字块的标记是否和主存编号（最高 $m-c$ 位）一致，若一致，则表示主存、Cache块已经建立对应关系，根据低 b 位地址从Cache取走要访问的字

Cache – 主存的地址映射

直接映射



每个缓存块 i 可以和若干个主存块对应

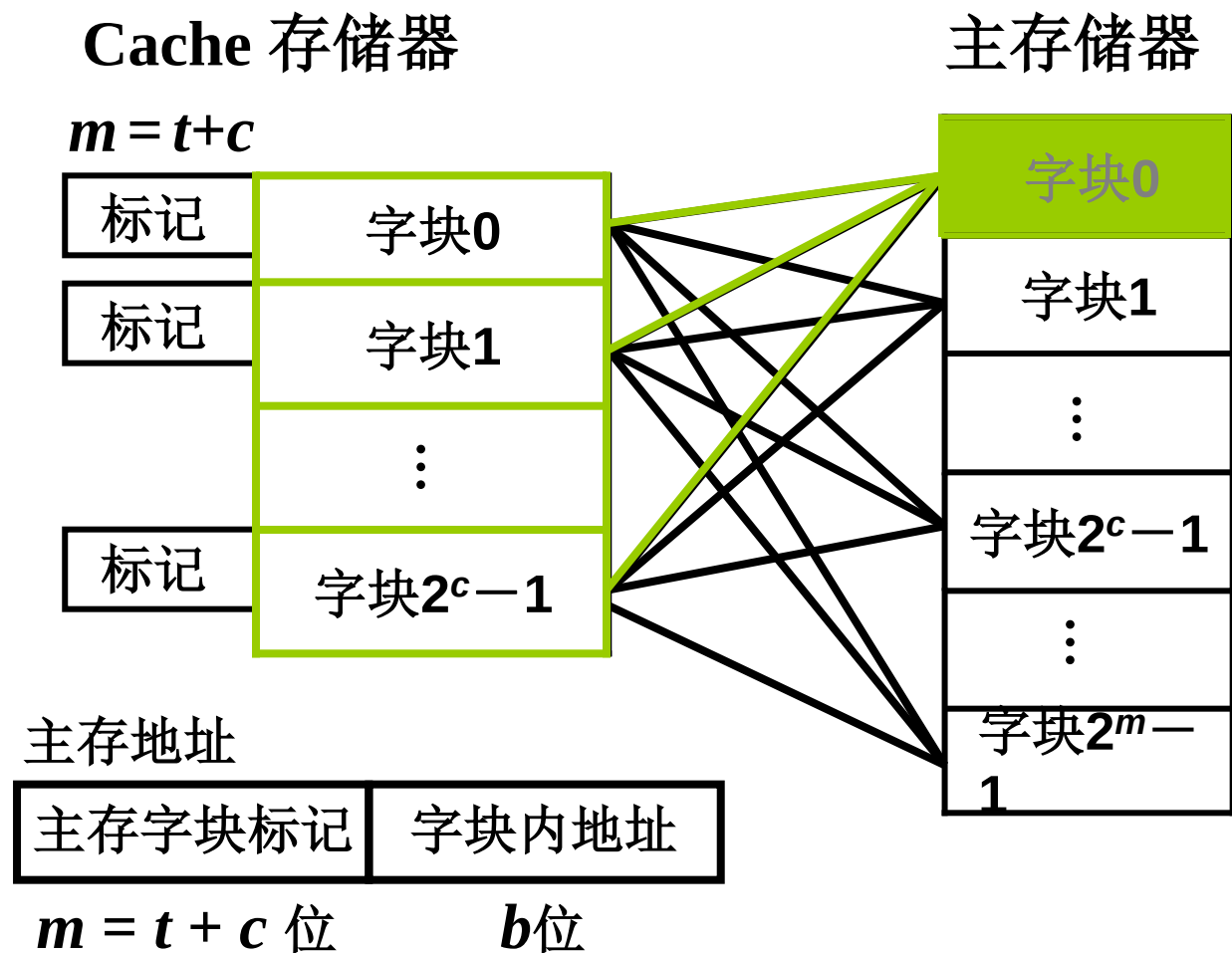
每个主存块 j 只能和一个缓存块对应



2. 全相联映射

- 最灵活、Cache空间利用最充分的映射方式
- 主存中某一字块可以映射到Cache中任一字块位置，是多对多的关系
- 因为一个Cache块可能存放任一个主存块，有M种可能，所以主存字块地址（m位）全部用作Cache块里的主存字块标记（可以对所有主存字块进行编号），CPU访存时，需要排查所有Cache块，查找某一Cache块是否当前对应将访问的主存块，这通过比较主存字块标记实现

全相联映射



主存 中的 任一 块 可以映射到 缓存 中的 任一 块

3. 组相联映射 直接映射和全相联映射的结合

- 把所有Cache块分为Q组，设每组有R块， $Q = C / R$
假定 $2^q = Q$ $2^r = R$ $2^c = C$ ，所以 $q=c-r$ （即分组地址二进制编码位数）
- 所有主存块和Q组之间是直接映射：某一主存块只能映射到固定的某一组 $i=j \bmod Q$
- 主存块和Cache分组内的所有块之间是全相联映射：对应某一分组的主存块可以映射到该组内任一个Cache块
- 地址映射：主存字块地址(共m位)分成两部分：低q位是Cache组地址，高m-q为主存字块标记
- n路组相联：Cache组内含有n个字块

组相联映射

组 Cache 共 Q 组，每组内两块 ($r = 1$)

0	标记	字块 0	标记	字块 1
1	标记	字块 2	标记	字块 3
	⋮	⋮	⋮	⋮
$2^{c-r}-1$	标记	字块 2^c-2	标记	字块 2^c-1

主存地址

主存字块标记	组地址	字块内地址
$s = t + r$ 位 $= m - c + r = m - q$ 位	$q = c - r$ 位	b 位
m 位		

主存储器

字块 0
字块 1
⋮
字块 $2^{c-r}-1$
字块 2^{c-r}
字块 $2^{c-r} + 1$
⋮
字块 2^{c-r+1}
⋮
字块 2^m-1

$i = j \bmod Q$ 直接相联映射

某一主存块 j 按模 Q 映射到 缓存 的第 i 组中的 任一块



小结

成本与活

直接 某一主存块只能固定映射到某一缓存块

全相联 某一主存块能映射到任一缓存块

组相联 某一主存块只能映射到某一缓存组中的任一块



Conclusion

- 高速缓存的原理：将CPU近期要用到的程序和数据提前从主存送到Cache，可以使CPU在一定时间内只访问速度较高的缓存
- 为了与Cache映射，把主存与缓存都分成若干块，主存和Cache之间以块为单位交换信息，每块内包含若干个存储字，主存和Cache的块大小一致
- 两者的字地址又可以分成两段：块号和块内地址

- 任何时候都有一些主存块处于缓存中
- 缓存块数远小于主存，一个缓存块不能只永远唯一对应一个主存块，因此缓存块中需设一标记，表示当前对应哪一个主存块。此标记相当于主存块的编号，是主存字地址的一部分
- 地址映射变换机构把CPU送来的主存字地址变换为对应的Cache字地址，主要是主存的块号和Cache的块号之间的转换，因为字的低位地址（块内地址，相对块起始地址的偏移量）相同

三、替换算法

- 当需要调入新的主存块进入Cache而其空间被占满时，需要替换掉Cache的数据，这需要按照一定的替换策略进行
- 直接映射时，一个主存块只能映射到一个Cache块，直接替换即可
- 在组相联和全相联映射的Cache中，主存块可以写入Cache的多个字块位置，就产生了选择替换掉哪一个Cache字块的问题，理想的替换算法是把未来很少用到或很久以后才用到的数据块替换出去

替换算法

1. 先进先出（FIFO）算法

优点：不需记录字块使用情况，容易实现

缺点：没有使用访存局部性原理，Cache命中率**低**

2. 近期最少使用（LRU）算法

优点：较好利用访存局部性原理

缺点：需随时记录各字块的使用情况

3. 随机法

随机确定被替换的数据块，非常简单



例题： 设主存容量为512KB， Cache容量为4KB， 每个字块为16个字， 每个字32位。

- (1)Cache地址有多少位， 可容纳多少字块？
- (2)主存地址有多少位， 可容纳多少字块？
- (3)设起始字块是第1字块， 在直接映射方式下， 主存第几块映射到Cache第5块？
- (4)画出直接映射方式主存地址字段中各段的位数。

解： (1) $4KB = 2^{12}$ Byte， 其地址包含12位二进制代码； 每个字块有 $16 \times 4 = 2^6$ Byte ，
Cache中字块数： $2^{12} / 2^6 = 2^6 = 64$

(2) $512\text{KB}=2^{19}$ Byte, 其地址包含19位二进制代码; 每个字块有 2^6 Byte, 则主存中字块数: $2^{19}/2^6=2^{13}=8\text{K}$

(3) 直接映射时, 因为 2^{13} 个主存块映射到 2^6 个Cache块, 因此映射到同一Cache块的主存块的数量为: $2^{13}/2^6=2^7$, 映射到Cache中第5块的主存块编号为:

$5, 64+5, 64*2+5, \dots, (2^7-1)*64+5$

没有特别指出时,
访存地址是字节
地址

(4) 直接映射方式主存地址字段分配

主存 字块标记 7位	缓存 字块地址 6位	字块内地址 6位
------------	------------	----------

字块标记长度是主存与Cache地址长度之差

例题：假设主存容量为 $512K \times 16$ 位，Cache容量为 4096×16 位，块长为4个16位的字，访存地址为字地址。

- (1) 直接映射方式，设计主存地址格式；
- (2) 全相联映射方式，设计主存地址格式；
- (3) 二路组相联映射，设计主存地址格式；
- (4) 若主存容量为 $512K \times 32$ 位，块长不变，在四路组相联方式设计主存地址格式。

(1) 主存容量为 $512K=2^{19}$ 字，主存字地址为19位；Cache容量为 $4K=2^{12}$ 字，Cache字地址为12位；每个字块有4个字，块内地址占2位， $b=2$ ；主存有 $2^{19}/2^2=2^{17}$ 个字块， $m=17$ ；Cache有 $2^{12}/2^2=2^{10}$ 个字块， $c=10$ ；直接映射方式，主存字块标记长度为 $17-10=19-12=7$ 位。地址格式：
7/10/2

指出当前主存块对应的缓存块，缓存块有 2^{10} 个，需要10位二进制代码进行编号

当前主存块的字块标记，因为有 2^7 个主存块对应1个缓存块，所以需要7位二进制代码进行编号

(2) 全相联映射方式，主存字地址中低两位是块内地址，高17位是主存字块标记；地址格式：17/2

当前主存块的地址编号的全部，也是其字块标记，因为这种映射方式中， 2^{17} 个主存块中的任何一块都可能映射到某一个缓存块

(3) 二路组相联映射方式，Cache的 2^{10} 个字块分组，每组含有两个字块，因此分组数为： $2^{10}/2=2^9$ ，组地址所占二进制位数： $q=9$ ，也可以这样算： $r=1$ ， $c=10$ ， $q=c-r=9$ ；主存字块标记长度： $m-q=17-9=8$ 位，或者 $19-2-9=8$ 位；地址格式：8/9/2

(4)主存容量改为： $512k \times 32\text{位} = 2^{20} \times 16\text{位}$ ，主存字地址为20位；字块长仍为 $4 \times 16\text{位}$ ，块内地址占两位，主存有 $2^{20}/2^2 = 2^{18}$ 个字块， $m=18$ ；

四路组相联映射，每个缓存组有4个字块，分组数为 $2^{10}/4 = 2^8$ ，组地址位数 $q=8$ ，因此主存字块标记为 $m-q=10$ 位，主存字地址格式10/8/2

例题：设某机主存容量为16MB，Cache容量为8KB，每字块有8个字，每字32位，设计一个四路组相联映射的Cache组织。

- (1) 画出主存地址字段中各段的位数
- (2) 设Cache初态为空，CPU依次从主存第0,1,2,...,99号读出100个字，并重复读此次序读10次，问命中率为多少
- (3) 若Cache速度为主存速度5倍，问有Cache比无Cache相比，速度提高多少倍
- (4) Cache系统效率为多少？

解：(1)主存容量为 $16\text{MB}=2^{24}$ 字节，主存字节地址为24位；Cache容量为 $8\text{KB}=2^{13}$ 字节，Cache字节地址为13位；每个字块有 $8 \times 4 = 32 = 2^5$ 个字节，块内地址占5位， $b=5$ ；主存有 $2^{24}/2^5=2^{19}$ 个字块， $m=19$ ；Cache有 $2^{13}/2^5=2^8$ 个字块， $c=8$ ；采用四路组相联映射，Cache每组有4个块，则Cache被分成了 $2^8/2^2=2^6$ 个组，组地址位数 $q=6$ ；主存字块标记位数 $m-q=19-6=13$ ；

地址格式：13/6/5

(2) CPU循环访问主存中地址连续的100个字，由于每个字块有8个字，这100个字连续分布在 $100/8=13$ 个主存字块中，Cache包含 2^6 个组，能够连续装入上述13个内存块；

开始时Cache为空，CPU读0号存储字时，需要访问主存，并将该字所在主存块调入Cache第0组，后面读1-7号存储字时Cache映射命中；以此类推，每8次读存储字只需在第一次访问主存，CPU首次读完连续100个字时只要13次访存，此后100个字所在主存块都位于Cache内，后面9次循环不需要访问主存，因此命中率为：

$$\frac{100*10-13}{100*10} = 0.987$$

(3) Cache的速度为主存的5倍，设Cache存取周期为 t ，则主存存取周期为 $5t$ ，若没有Cache全部访问时间是 $5t*1000$ ，有Cache的访问时间为 $t(1000-13)+5t*13$ ，则有Cache比没有Cache速度提高倍数为：

$$\frac{5t*1000}{t(1000-13)+5t*13} - 1 \approx 3.75$$

(4) 系统效率用Cache 访问时间和系统平均访问时间之比表示:

$$\frac{t}{0.987 * t + (1 - 0.987) * 5t} * 100\% = 95\%$$

一. 152页第25、26、28、29、32、33题。